Treća laboratorijska vježba: naučena detekcija objekata

U ovoj vježbi bavit ćemo se detekcijom objekata u slikama - jednim od najvažnijih zadataka raspoznavanja u računalnom vidu. Detekcija objekata podrazumijeva lokalizaciju i klasifikaciju objekata unutar slike. Lokalizaciju obično obavljamo procjenom koordinata pravokutnika koji najbolje opisuje objekt. Rani pristupi bavili su se gotovo isključivo binarnom detekcijom objekata (poput algoritma Viole i Jonesa), dok se u novije vrijeme podrazumijeva višerazredna detekcija. Problem ranih pristupa bio je izlučivanje dijeljenih značajki koje bi bile dovoljno dobre za raspoznavanje objekata različitih semantičkih razreda. Taj problem riješen je pojavom dubokih modela i mogućnosti učenja s kraja na kraj (engl. end-to-end learning).

Razvoj algoritama za detekciju objekata praćen je razvojem skupova podataka. Dugo vremena standard je bio skup podataka PASCAL VOC, koji razlikuje 20 razreda objekata. U posljednje vrijeme to mjesto je preuzeo skup podataka MS COCO, koji razlikuje 80 razreda objekata. Danas se već razmatra i raspoznavanje preko 1000 različitih semantičkih razreda na skupu LVIS.

Moderni pristupi za detekciju objekata bazirani su na dubokim modelima, a razlikujemo ih nekoliko. Najpopularniji pristupi pretpostavljaju tkz. sidrene okvire (engl. anchor boxes) u svim lokacijama mape značajki, te predviđaju relativne pomake i skale tih okvira kako bi najbolje opisali objekt. U ovu skupinu spadaju popularni pristupi poput Faster R-CNN-a, SSD-a, YOLO-a i sl. Drugi pristupi (npr. CornerNet) detektiraju objekte pomoću ključnih točaka koje uparuju na temelju predviđenih ugrađivanja. Transformeri su donijeli novi pristup koji je baziran na predviđanju skupa upita (engl. query set) iz čijih se reprezentacija izravno računa lokacija i klasifikacija objekta.

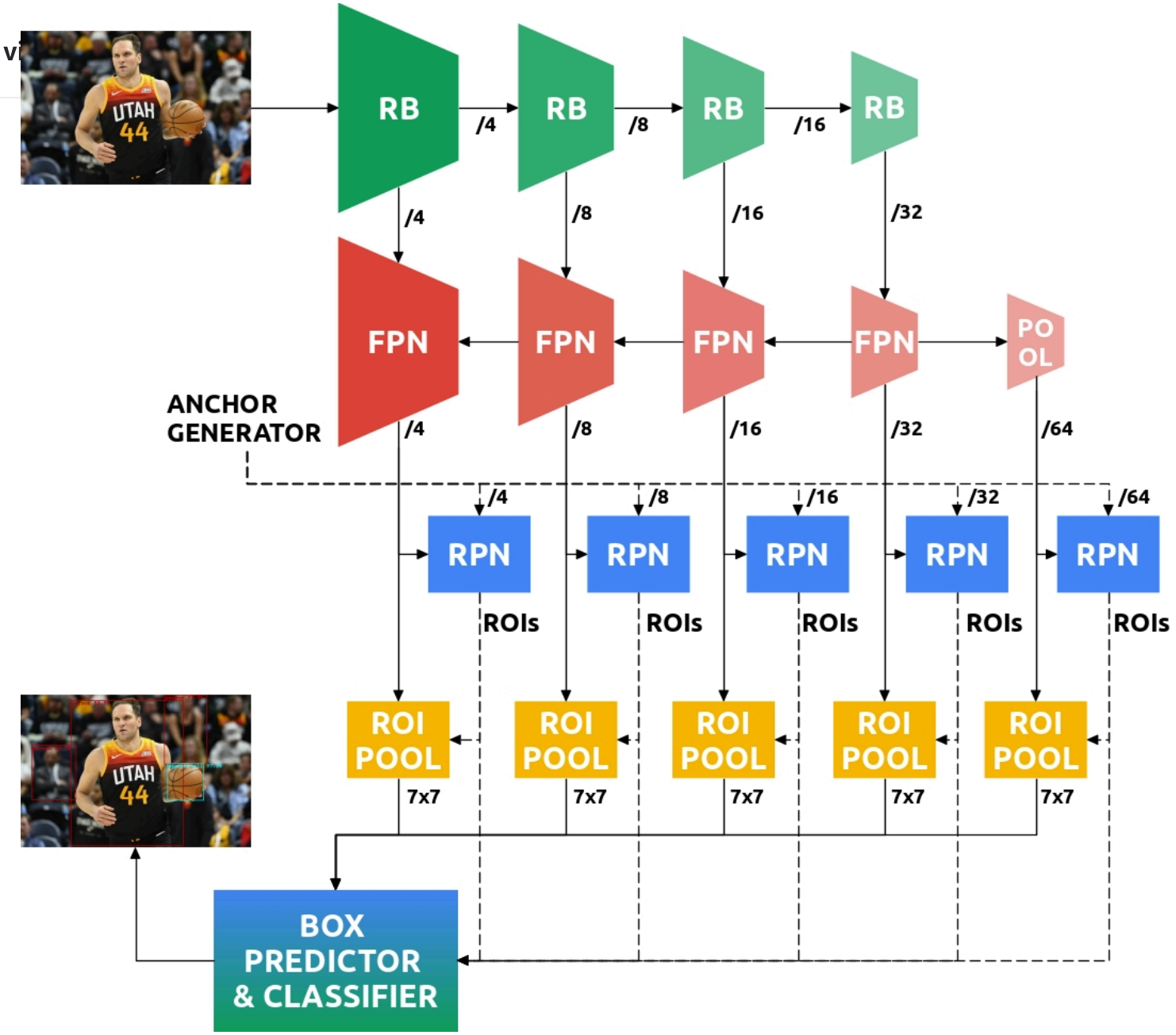
U ovoj vježbi pobliže ćemo se upoznati s modelom Faster R-CNN i to inačicom baziranom na piramidi značajki.

Faster R-CNN

Ključne komponente modela Faster R-CNN kojeg ćemo proučavati su:

- okosnica (engl. backbone) koja izračunava značajke slike,
- put za naduzorkovanje (engl. FPN, feature pyramid network) koji računa piramidu značajki približno jednake semantičke razine,
- mreža za predlaganje regija od interesa (engl. RPN, region proposal network),
- sloj sažimanja po regijama (engl. RoI pooling),
- te konačni sloj za klasifikaciju regija i fino podešavanje okvira.

Računalni vi



Slika 1. Arhitektura modela Faster R-CNN temeljena na piramidi značajki.

Na slici 1 prikazan je detaljan put zaključivanja modela Faster R-CNN iz kojeg je vidljiva interakcija između navedenih komponenti.

Obzirom da treniranje ovakvoga modela zahtijeva velike računalne resurse, u ovoj vježbi ćemo se fokusirati samo na unaprijedni prolaz već treniranog modela na skupu COCO. Vaš zadatak bit će dopuniti zadanu implementaciju modela Faster R-CNN. Upute će vas voditi kroz implementaciju komponentu po komponentu. Ispravnost svakog koraka vašeg rješenja provjeravat ćemo testovima koji će uspoređivati međurezultate vaše implementacije s rezultatima originalne implementacije.

1. Priprema okruženja

Za potrebe ove vježbe koristit ćemo Python 3 i sljedeće python pakete:

- pytorch
- torchvision
- pillow
- matplotlib
- numpy

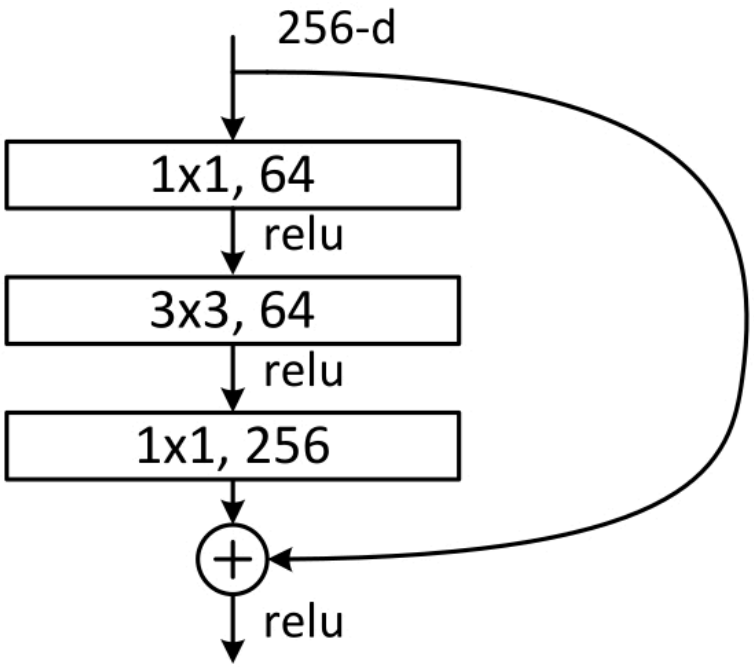
Nakon što ste se uvjerili da imate sve potrebne pakete, preuzmite kostur rješenja vježbe sa githuba. Zatim unutar repozitorija stvorite novi direktorij `data` i unutar njega raspakirajte sadržaj ovog direktorija. Preuzeta datoteka sadrži spremljene međurezultate unaprijednog prolaza razmatranog modela koji će se koristiti prilikom testova.

2. Okosnica

Put zaključivanja modela Faster R-CNN započinje izlučivanjem značajki uz pomoć okosnice. Okosnicu obično čini klasifikacijski model bez globalnog sažimanja i potpuno povezanog sloja, a koji je prethodno treniran na ImageNetu. Inicijalizacijom okosnice s ImageNet parametrima pospješujemo

proces treniranja modela za ciljani zadatak te smanjujemo potrebnu količinu gusto označenih slika. U slučaju ograničenih računalnih resursa, tijekom učenja detekcijskog modela parametri okosnice se mogu i smrznuti. Ipak, češći je slučaj da se oni tijekom treniranja fino ugađaju za detekciju objekata.

Naš azmatrani model za okosnicu koristi model ResNet-50 koji pripada obitelji modela s rezidualnim vezama. Osnovna gradivna jedinica modela ResNet-50 i njegove veće braće (ResNet-101, ResNet-152) prikazana je na slici 2. Za ovakvu rezidualnu jedinicu kažemo da ima usko grlo (engl. bottleneck) jer prva 1x1 konvolucija smanjuje broj kanala. To značajno smanjuje memorijski i računski otisak jer nakon nje slijedi skuplja 3x3 konvolucija. Konačno, broj kanala se ponovno napuhuje uz pomoć druge 1x1 konvolucije. Na slici se može primijetiti i preskočna veza koja ulaz u rezidualnu jedinicu pribraja rezultatu obrade iz konvolucijskih slojeva. Ona omogućuje modelu bolji protok gradijenata do ranijih slojeva te učenje jednostavnijeg rezidualnog mapiranja koje odgovara razlici između ulaza i “željenog” izlaza.



Slika 2. Rezidualna konvolucijska jedinica s uskim grlom.

Na slici 1 okosnica je prikazana nijansama zelene boje i sastoji se od četiri rezidualna bloka. Jednim blokom označavamo skup rezidualnih jedinica koje se izvršavaju nad značajkama iste prostorne rezolucije. Tako prvi rezidualni blok na svome izlazu daje značajke koje su četiri puta poduzorkovane u odnosu na ulaznu sliku. Na slici 1 je to označeno s "/4" na odgovarajućim strelicama. Slično, drugi rezidualni blok daje značajke koje su 8 puta poduzorkovane, treći 16, a četvrti 32 puta. U literaturi se često ove značajke referira s prefiksom "res" i brojem koji odgovara eksponentu potencije broja 2 koja označava razinu poduzorkovanja. Na primjer, izlazi prvog rezidualnog bloka označavaju se kao "res**2**", jer su 2^**2** = 4 puta poduzorkovani u odnosu na ulaznu sliku. Izlazi drugog rezidualnog bloka označavaju se kao "res**3**", jer su 2^**3** = 8 puta poduzorkovani u odnosu na ulaznu sliku. Slično, značajke preostala dva bloka označavat ćemo kao "res**4**" i "res**5**".

Zadaci

1. Obzirom da korištena okosnica očekuje normaliziranu sliku na ulazu, vaš prvi zadatak je implementirati funkciju za normalizaciju slike. Deklaraciju funkcije `normalize_img` pronaći ćete u datoteci `utils.py`. Funkcija na ulazu prima tenzor `img` koji je oblika (H, W, 3) gdje su H i W dimenzije slike. Funkcija treba vratiti normalizirani tenzor oblika (3, H, W). Normalizacija se sastoji od skaliranja na interval [0-1], oduzimanja srednje vrijednosti `image_mean` i dijeljenja sa standardnom devijacijom `image_std`. Srednja vrijednost i standardna devijacija oblika su (3). Ispravnost vaše implementacije možete provjeriti pozivom testne skripte "test_backbone.py". Pozicionirajte se u korjenski direktorij projekta te pozovite `python3 -m tests.test_backbone`.

2. Dopunite implementaciju modela ResNet u datoteci `resnet.py`. Prvo, prema slici 2 dopunite metodu `Bottleneck.forward` koja implementira unaprijedni prolaz kroz rezidualnu jedinicu s uskim grlom. Zatim, u metodi `ResNet._forward_impl` spremite izlaze iz rezidualnih blokova u rječnik `out_dict` pod ključevima "res2", "res3", "res4" i "res5". Primijetite da se u kodu prvi rezidualni blok označava s `layer1`, drugi s `layer2`, itd. Provjerite ispravnost vaše implementacije pokretanjem iste testne skripte kao u prethodnom zadatku.

Računalni vid

3 Koliko kanala imaju izlazi iz pojedinog rezidualnog bloka? Značajke kojeg rezidualnog bloka su semantički najbogatije, a koje prostorno najpreciznije?

3. Put za naduzorkovanje

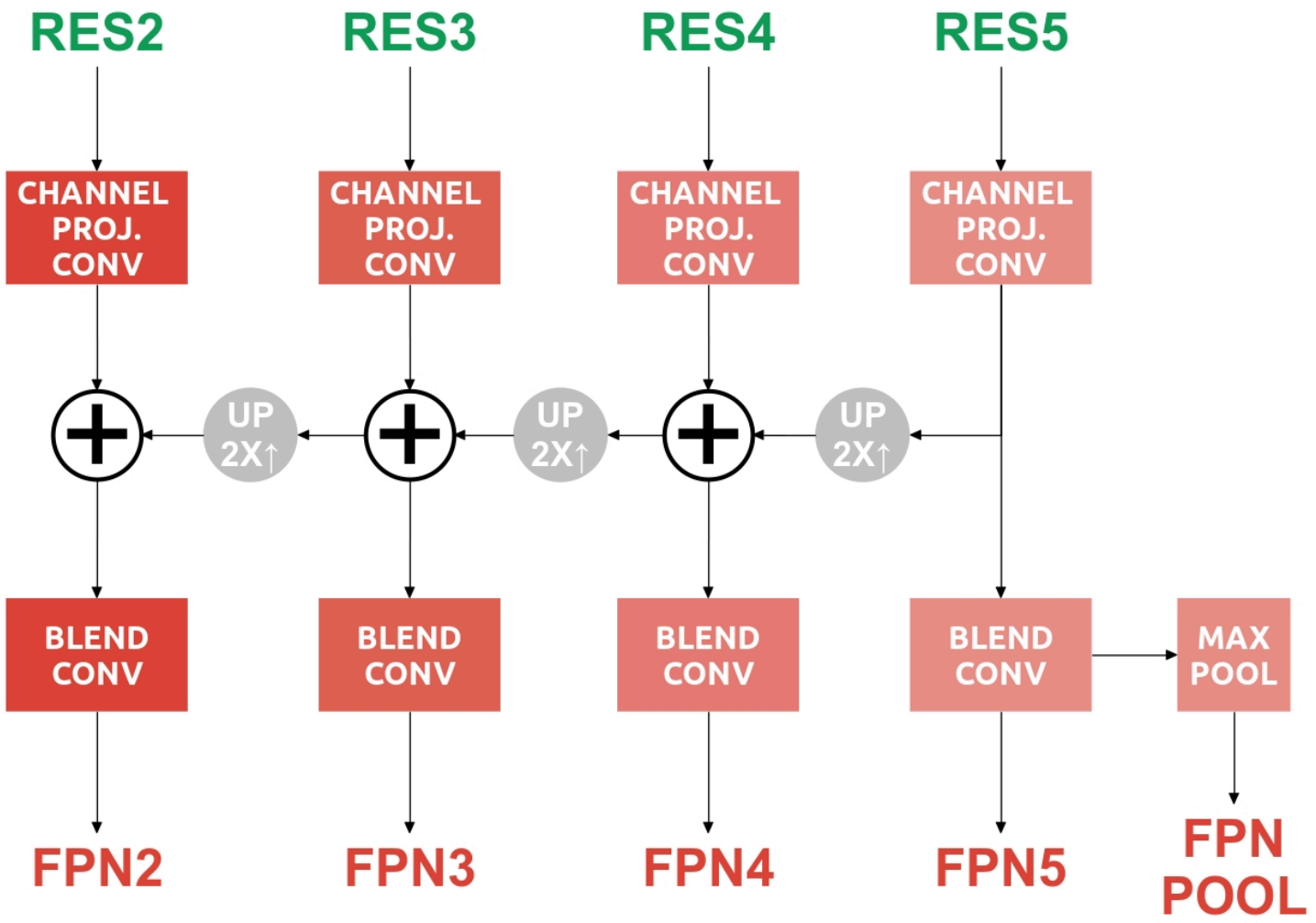
Općenito, zadaća puta za naduzorkovanje (engl. upsampling path) je izgradnja semantički bogate reprezentacije koja je istovremeno i prostorno precizna. Primijetite da niti jedan od izlaza okosnice ne zadovoljava oba spomenuta kriterija. Značajke iz kasnijih blokova okosnice su semantički bogatije, ali je njihova prostorna rezolucija manja. S druge strane, značajke iz ranijih blokova okosnice imaju finiju rezoluciju i zbog toga su prostorno preciznije, ali su semantički manje bogate. Stoga, ljestvičasti put za naduzorkovanje postupno gradi željenu reprezentaciju naduzorkovanjem semantički bogate reprezentacije i kombiniranjem s prostorno preciznijim značajkama. Različite inačice ove ideje prisutne su u mnogim modelima za gustu predikciju.

Posebne varijante puta naduzorkovanja prisutne su u detekciji objekata koji umjesto reprezentacije na jednoj razini grade rezolucijsku piramidu značajki. Razlog tomu je što bi oslanjanje na značajke isključivo jedne razine teško rezultiralo invarijantnosti na mjerilo. Primjerice, značajke niže rezolucije su dobre za detekciju velikih objekata, ali loše za detekciju malih jer bi se informacija o njihovoj prisutnosti mogla sasvim izgubiti uslijed poduzorkovanja. S druge strane, na većim rezolucijama bismo vjerojatno imali problema s detekcijom velikih objekata zbog ograničenog receptivnog polja konvolucijskih modela.

Stoga, već i prvi duboki modeli za detekciju objekata poput SSD-a razmatraju piramidu značajki (engl. feature pyramid). Oni izravno regresiraju opisujuće okvire iz značajki okosnice različitih rezolucija. Međutim, problem s takvim pristupom je u tome što značajke iz različitih faza okosnice se također nalaze na različitim semantičkim razinama. Tim problemom bavi se FPN koji koristi dodatni put za naduzorkovanje za izgradnju semantički bogate i ujednačene rezolucijske piramide značajki.

Naša inačica modela Faster R-CNN također koristi FPN. Na slici 1 put naduzorkovanja označen je crvenom bojom. Različite nijanse crvene boje ukazuju na činjenicu da moduli za svaku razinu piramide

koriste različite parametre. Svaki modul za naduzorkovanje ima dvije konvolucijske jedinice. Jednu koja se primjenjuje na odgovarajuće značajke iz okosnice (često nazivane preskočnim ili lateralnim vezama) kako bi se izjednačio broj kanala s putem naduzorkovanja. U literaturi se ove konvolucije stoga često zovu i kanalnim projekcijama (engl. channel projection). Druga konvolucijska jedinica se primjenjuje na zbroj preskočne veze i naduzorkovane sume iz prethodne razine za izračun konačne reprezentacije na toj razini piramide. Detaljniji prikaz razmatranog puta nadozorkovanja nalazi se na slici 3.



Slika 3. Detaljniji prikaz puta naduzorkovanja koji gradi piramidu značajki.

Zadaci

1 U datoteci `utils.py` implementirajte modul `ConvNormActBlock` čiji se unaprijedni prolaz sastoji redom od konvolucijskog sloja, te opcionalno aktivacijske funkcije i opcionalno normalizirajućeg sloja. Primijetite da modul nasljeđuje razred `nn.Sequential` koji redom provodi zadanu listu slojeva. Za dodavanje sloja u listu možete koristiti metodu `self.append` . Argument `padding` koji

kontrolira nadopunjavanje prilikom konvolucije potrebno je postaviti tako da se očuvaju prostorne dimenzije ulaza.

2. U datoteci `fpn.py` implementirajte unaprijedni prolaz puta za naduzorkovanje u metodi `FeaturePyramidNetwork.forward`. Unaprijedni prolaz implementirajte prema slici 3. Pripazite na komentare napisane u kodu.

3. Testirajte svoju implementaciju puta nadozurkovanja pokretanjem naredbe `python3 -m tests.test_fpn`.

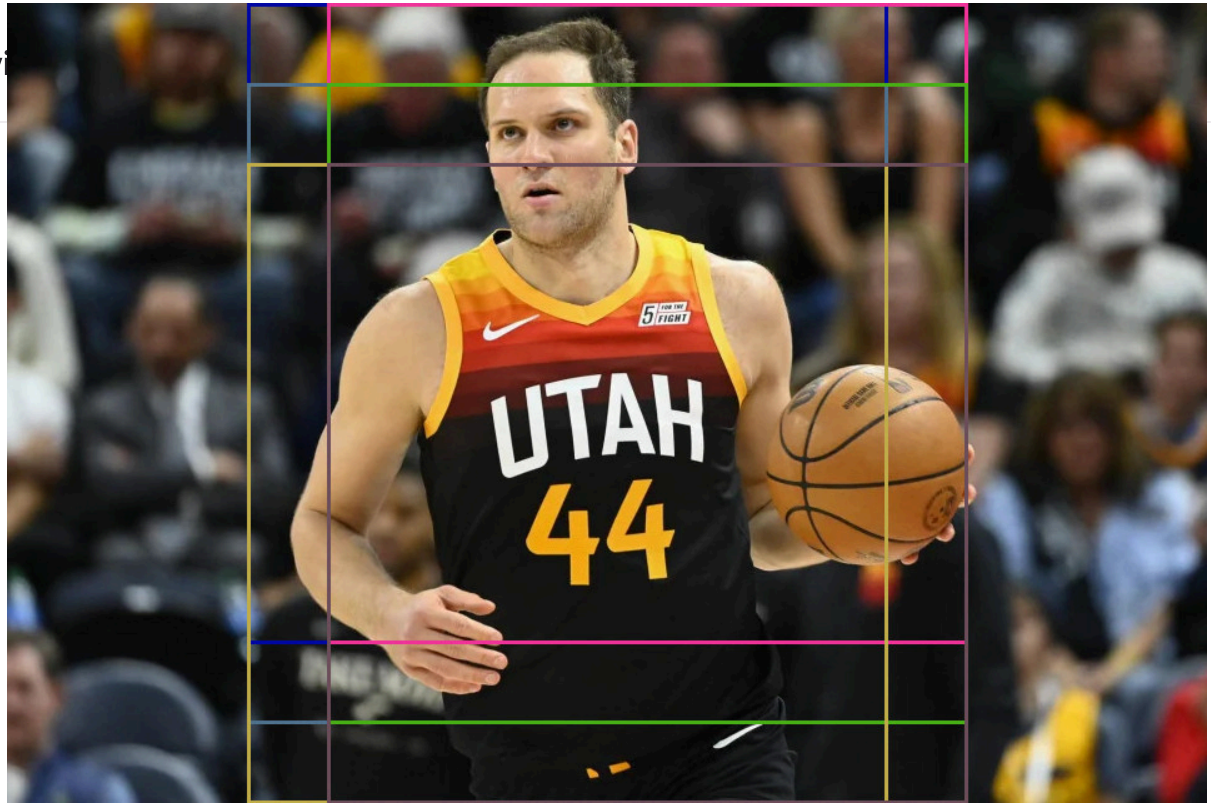
4. Mreža za predlaganje regija od interesa (RPN)

Zadaća mreže za predlaganje regija od interesa je izdvojiti pravokutne regije unutar kojih bi se mogao nalaziti neki objekt. Taj zadatak sveden je na binarnu klasifikaciju sidrenih okvira u pozitive i negative. Negativi se odbacuju jer su to okviri koji ne sadrže objekte, dok se pozitivi parametriziranom transformacijom mijenjaju na način da preciznije uokviruju ciljani objekt. Primijetite da mreža za predlaganje regija od interesa ne razlikuje semantičke razrede objekata (u literaturi se često koristi engl. izraz class-agnostic). Njena zadaća je samo procijeniti bi li se neki objekt mogao nalaziti unutar razmatranog sidrenog okvira ili ne. Pozitivni okviri se transformiraju parametrima t_x, t_y koji kontroliraju horizontalni i vertikalni pomak centra okvira te parametrima t_w, t_h koji kontroliraju horizontalno i vertikalno skaliranje okvira. Za spomenute parametre vrijede sljedeće jednadžbe:

$$t_x = \frac{x - x_a}{w_a}$$
$$t_y = \frac{y - y_a}{h_a}$$
$$t_w = \log \frac{w}{w_a}$$
$$t_h = \log \frac{h}{h_a}$$

gdje x_a, y_a, w_a, h_a predstavljaju koordinate centra, širinu i visinu sidrenog okvira, a x, y, w, h predstavljaju koordinate centra, širinu i visinu ciljanog objekta.

Iz slike 1 vidljivo je da se mreža za predlaganje regija od interesa primjenjuje na svaku razinu piramide značajki. Jednaka nijansa plave boje za svaku razinu piramide sugerira dijeljenje parametara. Pored značajki ulaz u RPN čine i sidreni okviri koji se također generiraju ovisno o razini piramide. Konkretno, generator sidrenih okvira smješta okvire u svakom pikselu razmatranog tenzora značajki, a njihova veličina ovisi o razini piramide. Tako se na razini piramide najveće rezolucije pretpostavljaju sidreni okviri najmanje osnovne veličine jer na toj razini želimo detektirati male objekte. Obrnuto, na razini piramide najmanje rezolucije nalaze se sidreni okviri najveće osnovne veličine jer na toj razini želimo detektirati velike objekte. Važan detalj je da generator sidrenih okvira ne pretpostavlja samo jedan okvir po lokaciji, nego više njih, a razlikuju se po omjeru visine i širine kako bi se mogli detektirati objekti različitih oblika. Konkretno, u modelu kojeg mi razmatramo osnovne veličine sidrenih okvira su [32, 64, 128, 256, 512] za redom razine piramide [fpn2, fpn3, fpn4, fpn5, fpn6], a razlikuju se omjeri visine i širine [1:1, 1:2, 2:1]. RPN također zasebno razmatra svaki od pretpostavljenih sidrenih okvira na nekoj lokaciji. To znači da RPN klasifikator predviđa onoliko mapa značajki koliko ima pretpostavljenih sidrenih okvira u svakoj lokaciji. Slično tome, RPN regresor parametara transformacije predviđa 4 puta više mapa značajki negoli ima pretpostavljenih sidrenih okvira u svakoj lokaciji. Na slici ispod prikazani su sidreni okviri čiji je omjer presjeka i unije s okvirom igrača na slici veći od 0.65.



Slika 4. Pretpostavljeni sidreni okviri koji se s okvirom košarkaša na slici preklapaju s omjerom presjeka i unije većim od 0.65

Spomenimo još da RPN ne propušta sve pozitivne okvira kroz unaprijedni prolaz. Nakon što se odbace negativi i na pozitive primjeni predviđena transformacija, pristupa se filtriranju. Prvo se odbacuju okviri čija je površina manja od zadane, a zatim i oni koji imaju vjerojatnost prisutnosti objekta nižu od zadanog praga. Nakon toga se potiskuju nemaksimalni odzivi, odnosno okviri koji imaju visoko preklapanje s nekim drugim pouzdanim okvirom. Konačno, propušta se samo 1000 okvira s najvećom vjerojatnošću. Ovo filtriranje značajno ubrzava unaprijedni prolaz kroz mrežu. Na slici ispod prikazani su okviri koje je predložio RPN, a imaju omjer presjeka i unije s okvirom igrača na slici veći od 0.65.



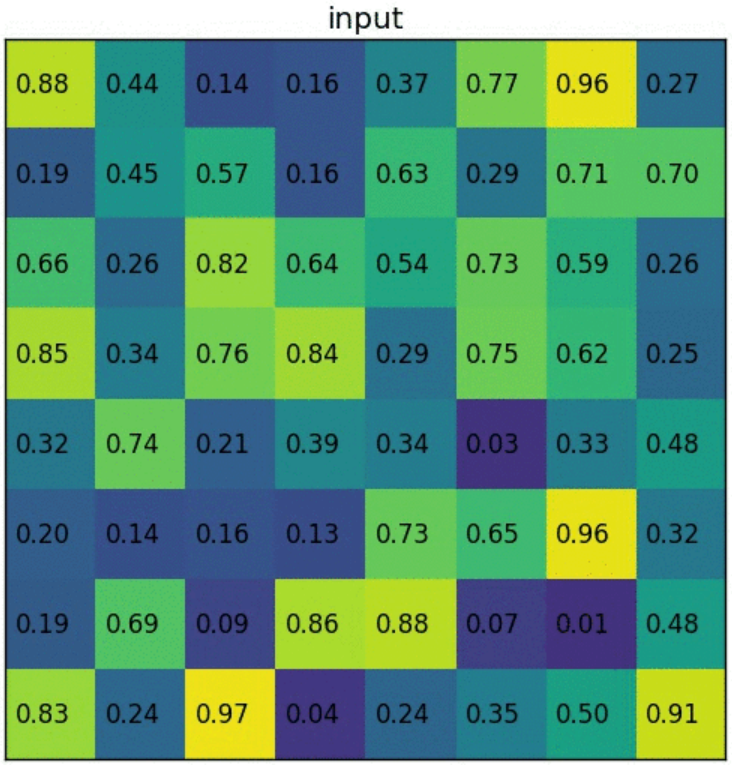
Slika 5. Regije od interesa predložene od strane RPN-a koje se s okvirom košarkaša na slici preklapaju s omjerom presjeka i unije većim od 0.65.

Zadaci

1. U datoteci `rpn.py` dovršite inicijalizaciju klasifikatora i regresora RPN-a u modulu RPNHead.
2. U datoteci `utils.py` dovršite implementaciju funkcije `decode_boxes` koja primjenjuje predviđenu transformaciju na sidrene okvire. Implementaciju testirajte naredbom `python3 -m tests.test_decode_boxes .`

5. Sažimanje po regijama (ROIPool)

Sažimanje po regijama (engl. Region of Interest Pooling, ROIPool) izlučuje reprezentaciju fiksne veličine za sve regije od interesa koje predlaže RPN. Sažimanje po regijama prvo odredi područje u razmatranom tenzoru značajki koje odgovara regiji od interesa. Zatim se to područje dijeli na manja podpodručja približno jednake veličine, a njihov broj određen je parametrom modela. Zatim se iz značajke svakoga područja sažimlju prema zadanoj funkciji (npr. maksimumom). Ovaj proces ilustriran je na gif-u ispod za tenzor značajki sa samo jednim kanalom i zadanom veličinom izlaza 2x2.



Slika 6. Ilustracija operacije sažimanja po regijama od interesa. Izvor: Deepsense.ai.

U našem razmatranom modelu veličina izlaza ROIPool-a je 7x7, ali to ne znači da se svaki okvir predstavlja sa samo 49 značajki. Taj broj treba još pomnožiti s brojem kanala razmatranog tenzora

značajki što odgovara broju kanala FPN-a, a to je 256. Dakle, svaki okvir predstavljen je s 7x7x256=12544 značajki. Spomenimo još da novije inačice dvoprolaznih modela obično koriste napredniji algoritam ROIAlign koji koristi interpolaciju umjesto kvantizacije za određivanje rubnih vrijednosti.

6. Semantička klasifikacija i fino ugađanje okvira

Završni modul našeg razmatranog modela na ulazu prima sažetu reprezentaciju i koordinate svih regija od interesa koje je predložio RPN. Zadaća ovog modula je semantička klasifikacija pristiglih okvira i predviđanje paramatera još jedne transformacije za fino ugađanje okvira na željene objekte. Razlika u odnosu na RPN je što ovaj modul klasificira okvire u jedan od semantičkih razreda ili pozadinu. Postojanje razreda koji predstavlja pozadinu omogućuje modelu da i u ovome koraku odbaci neke okvire koje smatra negativima.

Zanimljiv detalj je da se za svaki okvir predviđaju odvojeni parametri transformacije za svaki od semantičkih razreda. Možemo zamisliti kao da svaki okvir umnožimo onoliko puta koliko imamo razreda i za svaki od njih predvidimo parametre transformacije. Ovo omogućuje detekciju preklapajućih objekata različitih razreda.

Ovaj modul prvo računa dijeljenu reprezentaciju za klasifikacijsku i regresijsku glavu uz pomoć dva potpuno povezana sloja. Zatim se na tu reprerzentaciju primjenjuju još dva potpuno povezana sloja: jedan za klasfikaciju, a drugi za regresiju semantički ovisnih parametara transformacije.

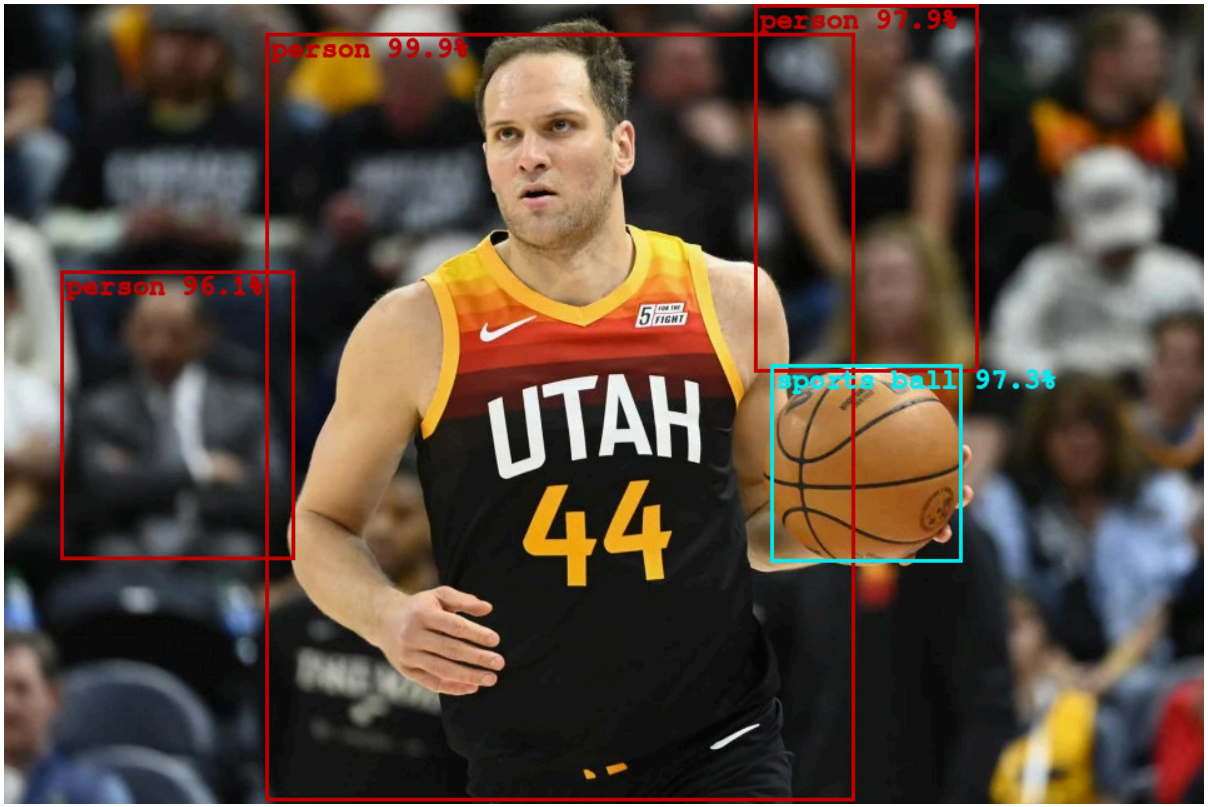
Zadaci

1. U datoteci `faster.py` dovršite implementaciju funkcije `forward` u modulu `TwoMLPHead`.
2. U datoteci `faster.py` dovršite inicijalizaciju klasifikacijske i regresijske glave u modulu `FastRCNNPredictor`.
3. U datoteci `run_faster.py` implementirajte iscrtavanje rezultate detekcije za sve okvire čija je pouzdanost veća od 0.95.

4 Pokrenite program naredbom `python3 run_faster.py` i provjerite je li detekcija uspješna.

Računalni vid

Očekivani rezultat izvođenja programa `run_faster.py` prikazan je na slici ispod.



Slika 7. Rezultat izvođenja modela Faster R-CNN treniranog na skupu COCO.